FET · Optativa II · Semana 7 · Grupo 6

Plan mínimo de pruebas

API REST CRUD PokeAPI · Ronda de verificación local

Hernan Felipe Patiño Rico · Juan Jose Sabogal Garcia · Orlando Rojas Narvaez

1. Objetivo y alcance

Verificar creación, consulta, edición, eliminación y persistencia de registros; catálogo Gen 1, transformación de PokeAPI, sincronización y errores HTTP. Confirmar que entradas inválidas no alteran los datos. Fuera de alcance: carga, concurrencia entre procesos, auditoría de seguridad y cobertura completa de la interfaz.

2. Prioridad por riesgo

Escala: probabilidad (P) e impacto (I) de 1 a 3. Riesgo = P × I. Alto: 6-9; medio: 3-4; bajo: 1-2. Estimaciones del plan, no frecuencias medidas.

Riesgo	P	I	P×I	Casos / prioridad
API inaccesible o CRUD esencial roto	3	3	9	S01-S05 / primero
Datos inválidos guardados o edición parcial dañada	3	3	9	R04-R06, R10 / alta
Dependencia externa o sincronización fallida	2	3	6	R08-R09 / alta
Catálogo o mapeo incorrectos; recursos ausentes	2	2	4	R01-R03, R07 / media

3. Ambiente, datos y herramientas

Windows, Node.js 24.21.0, Express y aserciones nativas de Node. Cada caso inicia la API en un puerto libre con almacenamiento temporal vacío y PokeAPI simulada. Sincronización inicial desactivada. Datos: Ditto (seis valores 48), Pikachu (35, 55, 40, 50, 50, 90), 151 entradas sintéticas y un registro local. Límites válidos: 1 y 999, según el formulario del proyecto. La comprobación real usa PokeAPI pública por separado.

4. Suites y ejecución

Smoke, 5 casos: disponibilidad, crear, consultar, editar y persistir, eliminar. Regresión, 10 casos: mapeo, Pokémon inexistente, paginación, obligatorios, límites/tipos, edición inválida, ID inexistente, sincronización, fallos externos y JSON inválido. Cada caso tiene criterio CA01-CA15 y pasos en CASOS.md.

Ejecutar npm run test:smoke; si pasa, npm run test:regression. npm test ejecuta los 15 para diagnóstico completo. Ejecutar npm run test:live para tres consultas reales adicionales.

5. Entrada, detención y salida

Entrada: dependencias instaladas, API de prueba iniciable, datos aislados y esperados definidos. Detener promoción: cualquier smoke FALLA/BLOQUEADO, pérdida de datos o defecto alto abierto. Conservar evidencia; corregir y repetir los casos afectados y ambas suites. El diagnóstico puede continuar para registrar otros fallos.

Salida GO: 5/5 smoke y 10/10 regresión PASA, sin bloqueos ni defectos altos abiertos dentro del alcance, con evidencia. Para demostrar integración en línea se exige además 3/3 consultas reales PASA. NO-GO: cualquier condición incumplida. PASA = coincide; FALLA = difiere; BLOQUEADO = ambiente/dependencia impide comprobar.

6. Evidencia y control de ronda

Guardar caso, resultado, solicitud/respuesta, observación, fecha UTC, versión de Node y huellas del código en evidencias/. Registrar defectos, repetir tras la corrección y cerrar con decisión justificada. El grupo revisa resultados y prepara la sustentación; no se atribuyen ejecuciones individuales.

Fuente académica: “OptativaII_Semana_7_Estrategia_de_pruebas_y_plan_m_nimo_v2.pptx”, diapositivas 4-9, Diego Fernando Vargas Ramírez. Código base: c468b40f1258fd01b1228544065f1a123eef2825. Plan de una página A4.